SIMULACRO DE PREPARACIÓN – MÓDULO 6

Arquitectura Hexagonal + JPA Avanzado + Seguridad JWT + Observabilidad + Microservicios + Docker + Pruebas

Modalidad: Práctica autónoma de preparación

Nivel: Backend Java / Spring Boot

Propósito: Fortalecer las competencias técnicas antes de la prueba de desempeño.

1. Contexto del proyecto

Caso de estudio: StockBridge – Sistema de Gestión de Inventario y Solicitudes Internas

StockBridge es una empresa dedicada a la distribución de equipos tecnológicos, accesorios y suministros empresariales. Actualmente cuenta con diferentes bodegas y sedes, pero la gestión de inventarios y solicitudes internas se realiza mediante archivos de Excel, correos electrónicos y registros manuales.

Esta situación ha generado diferentes dificultades:

- Diferencias entre las existencias físicas y los registros del sistema.
- Solicitudes de productos sin seguimiento.
- Ausencia de controles para evitar entregas superiores al inventario disponible.
- Falta de trazabilidad sobre movimientos de inventario.
- Dificultad para identificar responsables de las operaciones.

Procesos de autorización manuales.

Falta de indicadores y herramientas de monitoreo.

La compañía desea desarrollar una solución tecnológica que permita administrar productos, controlar existencias, gestionar solicitudes internas y consultar un servicio externo encargado de simular la disponibilidad de abastecimiento.

El equipo de desarrollo deberá proponer una solución utilizando buenas prácticas de ingeniería de software y los conocimientos adquiridos durante el módulo.

2. Objetivo del simulacro

Desarrollar una aplicación backend que permita gestionar el inventario de una empresa y sus solicitudes de productos, aplicando los conceptos estudiados durante las semanas 2 a 6.

La solución deberá estar organizada en dos servicios:

1. inventory-management-service

Servicio principal encargado de gestionar productos, usuarios, existencias, solicitudes y movimientos de inventario.

2. supplier-availability-service

Servicio independiente que simula la consulta de disponibilidad de productos en proveedores externos.

El propósito no es replicar una implementación específica, sino demostrar comprensión de arquitectura, persistencia, seguridad, integración, pruebas y despliegue.

3. Requerimientos funcionales

3.1. Gestión de productos

El sistema debe permitir:

- Registrar productos con código, nombre, descripción, categoría, precio y cantidad disponible.
- Consultar productos registrados.
- Actualizar información.
- Activar o desactivar productos.
- Consultar productos con existencias inferiores a un mínimo establecido.

Reglas de negocio:

- No pueden existir dos productos con el mismo código.
- El precio debe ser mayor que cero.
- Las existencias no pueden ser negativas.
- Los productos inactivos no pueden solicitarse.

3.2. Gestión de solicitudes internas

Los usuarios de la empresa podrán solicitar productos del inventario.

Cada solicitud debe registrar:

- Usuario solicitante.

- Producto solicitado.
- Cantidad requerida.
- Fecha de creación.
- Estado.
- Observación.

Estados sugeridos:

- PENDIENTE
- APROBADA
- RECHAZADA
- ENTREGADA
- CANCELADA

El equipo deberá establecer las transiciones válidas entre estados y evitar operaciones inconsistentes.

Ejemplo de negocio:

Un colaborador solicita cinco monitores. El sistema verifica si el producto está activo y si existen suficientes unidades disponibles.

Si la solicitud es aprobada, debe ser posible registrar posteriormente la entrega y descontar las unidades correspondientes del inventario.

Si no existe disponibilidad suficiente, el sistema deberá permitir consultar el servicio externo de abastecimiento.

3.3. Integración con proveedores

El segundo servicio simulará la disponibilidad de productos.

Endpoint sugerido:

POST /api/suppliers/check-availability

Ejemplo de petición:

{ "productCode": "MON-001", "quantity": 10, "category": "MONITORES"}

Ejemplo de respuesta:

{ "supplierCode": "SUP-003", "available": true, "availableQuantity": 25, "estimatedDeliveryDays": 3}

El servicio deberá responder de manera consistente para una misma combinación de datos.

Reto de análisis: ¿Cómo se debe comportar el servicio principal cuando el proveedor externo no responde, devuelve un error o supera el tiempo máximo de espera?

4. Arquitectura Hexagonal

El servicio principal deberá utilizar los principios de arquitectura hexagonal.

Se espera que los coders puedan identificar y separar:

- Entidades y reglas del dominio.
- Casos de uso.
- Puertos de entrada.
- Puertos de salida.

Adaptadores REST.

Adaptadores de persistencia.

Adaptadores de comunicación con servicios externos.

Aspectos para estudiar:

Diferencia entre entidad de dominio y entidad JPA.

Responsabilidades de un caso de uso.

Inversión de dependencias.

Uso de interfaces para desacoplar infraestructura.

Transformación de datos mediante MapStruct.

Ventajas de mantener el dominio independiente de Spring.

Pregunta orientadora: Si mañana la empresa cambia PostgreSQL por otra base de datos, ¿qué componentes deberían modificarse y cuáles deberían permanecer intactos?

5. Persistencia y JPA avanzado

Implementar persistencia relacional con Spring Data JPA e Hibernate.

Relaciones sugeridas:

Un usuario puede realizar múltiples solicitudes.

Un producto puede aparecer en múltiples solicitudes.

Un producto puede tener múltiples movimientos de inventario.

Investigar y aplicar:

- @OneToMany
- @ManyToOne
- FetchType.LAZY
- @EntityGraph
- Consultas JPQL.
- Paginación y ordenamiento.
- Transacciones con @Transactional.
- Migraciones de base de datos con Flyway.

Reto técnico: Garantizar que la entrega de productos y el registro del movimiento de inventario se realicen correctamente dentro de una misma transacción.

Pregunta orientadora: ¿Qué sucedería si se descuentan diez unidades del inventario, pero falla el registro del movimiento?

6. Seguridad con Spring Security y JWT

Implementar autenticación mediante JWT y control de acceso basado en roles.

Roles sugeridos:

- ROLE_EMPLEADO: registrar y consultar sus propias solicitudes.
- ROLE_BODEGA: gestionar entregas y movimientos de inventario.

ROLE_ADMIN: administrar productos, usuarios y configuraciones.

Endpoints de autenticación:

POST /auth/register

POST /auth/login

Consideraciones:

Contraseñas protegidas mediante PasswordEncoder.

Autenticación stateless.

Validación de tokens.

Restricción de endpoints.

Control de acceso a recursos propios.

Respuestas apropiadas para errores de autenticación y autorización.

Pregunta orientadora: Si un empleado conoce el identificador de una solicitud realizada por otro empleado, ¿cómo se evita que pueda consultarla?

7. Validaciones y manejo de errores

Aplicar Bean Validation y un mecanismo centralizado de manejo de excepciones.

Situaciones que deben contemplarse:

Productos inexistentes.

Códigos duplicados.

Cantidades inválidas.

Solicitudes no autorizadas.

Inventario insuficiente.

Transiciones de estado incorrectas.

Errores de integración con el proveedor externo.

Implementar respuestas de error utilizando ProblemDetail, incluyendo información útil para identificar el problema.

Reto: Diferenciar correctamente errores de validación, reglas de negocio, autenticación, autorización y fallas internas.

8. Pruebas automatizadas

La solución debe contar con pruebas que permitan verificar su comportamiento.

Pruebas unitarias

Utilizar JUnit 5 y Mockito para validar las reglas de negocio y los casos de uso.

Escenarios sugeridos:

Registrar un producto válido.

Rechazar un producto duplicado.

Impedir solicitudes con cantidades inválidas.

Evitar entregas sin inventario suficiente.

Simular la respuesta del proveedor externo.

Pruebas de integración

Utilizar Spring Boot Test y MockMvc para comprobar:

Endpoints REST.

Persistencia.

Autenticación.

Restricciones por roles.

Flujo de solicitudes.

Testcontainers

Investigar cómo ejecutar pruebas reproducibles utilizando una base de datos en contenedor.

Pregunta orientadora: ¿Por qué una prueba que utiliza Mockito no reemplaza necesariamente una prueba de integración con una base de datos real?

9. Observabilidad

Implementar Spring Boot Actuator y Micrometer.

Endpoints de referencia:

/actuator/health

/actuator/info

/actuator/metrics

Indicadores sugeridos:

Cantidad de solicitudes creadas.

Solicitudes aprobadas y rechazadas.

Productos con inventario bajo.

Tiempo de respuesta de las consultas externas.

Errores durante las operaciones.

Incluir logging que permita identificar operaciones y errores relevantes.

Pregunta orientadora: ¿Cómo identificarías si las solicitudes están fallando por un problema de base de datos o por indisponibilidad del servicio externo?

10. Docker y comunicación entre servicios

La solución debe poder ejecutarse mediante contenedores.

Investigar e implementar:

Dockerfile multi-stage.

Construcción de aplicaciones Java con Maven.

Imágenes de ejecución optimizadas.

Variables de entorno.

Redes internas de Docker.

- Docker Compose.
- Comunicación HTTP entre contenedores.

Servicios esperados:

- inventory-management-service
- supplier-availability-service
- Base de datos relacional

Reto técnico: Ejecutar todo el sistema utilizando un único comando de Docker Compose.

11. Retos adicionales de preparación

Para quienes terminen los requerimientos principales, se proponen los siguientes desafíos:

Reto 1 – Concurrencia

Dos empleados intentan solicitar simultáneamente las últimas cinco unidades de un producto.

¿Cómo evitar que el inventario termine con cantidades negativas?

Reto 2 – Rendimiento

Un endpoint devuelve 100 solicitudes con información de sus productos y usuarios.

¿Cómo identificar y solucionar un posible problema N+1?

Reto 3 – Resiliencia

El servicio de proveedores deja de responder durante varios segundos.

¿Cómo evitar que esa situación afecte indefinidamente las solicitudes del servicio principal?

Reto 4 – Seguridad

Un usuario modifica manualmente un JWT o intenta acceder a recursos administrativos.

¿Qué controles deben impedir estas operaciones?

Reto 5 – Trazabilidad

Una entrega falla durante la actualización del inventario.

¿Cómo verificar que los datos permanezcan consistentes y que el error quede registrado?

12. Entregables del simulacro

Cada coder deberá preparar:

1. Repositorio GitHub con el código fuente.
2. Dos servicios funcionales.
3. Arquitectura hexagonal aplicada al servicio principal.
4. Base de datos con migraciones.
5. Autenticación y autorización con JWT.
6. Integración HTTP entre servicios.

7. Pruebas unitarias y de integración.
8. Configuración Docker Compose.
9. Documentación Swagger/OpenAPI.
10. README con instrucciones de ejecución, decisiones técnicas y diagramas.

13. Guía de autoevaluación

Antes de dar por terminado el simulacro, cada coder deberá poder responder:

- ¿Puedo explicar el recorrido de una petición desde el controlador hasta la base de datos?
- ¿Mi dominio depende directamente de Spring o Hibernate?
- ¿Comprendo por qué se utilizan puertos y adaptadores?
- ¿Sé cuándo utilizar @Transactional?
- ¿Puedo identificar y corregir consultas N+1?
- ¿Entiendo cómo se genera y valida un JWT?
- ¿Mis endpoints verifican roles y propiedad de los recursos?
- ¿Puedo simular servicios externos con Mockito?
- ¿Puedo ejecutar pruebas de integración con Testcontainers?
- ¿Sé interpretar las métricas de Actuator?
- ¿Puedo levantar todos los servicios mediante Docker Compose?

¿Puedo explicar y justificar mis decisiones de arquitectura?

14. Recomendaciones para los coders

Este simulacro es una herramienta de preparación, no una solución de la prueba de desempeño.

Se recomienda:

Repasar los conceptos antes de comenzar a programar.

Diseñar primero el dominio y los casos de uso.

Construir la solución progresivamente.

Realizar commits frecuentes.

Escribir pruebas mientras se desarrollan las funcionalidades.

Consultar la documentación oficial.

Utilizar herramientas de inteligencia artificial como apoyo para comprender conceptos, depurar errores y contrastar decisiones, evitando depender de soluciones generadas sin entenderlas.

Practicar la explicación técnica de cada componente implementado.

El objetivo no es únicamente conseguir que el proyecto funcione, sino comprender cómo está construido, por qué se tomaron determinadas decisiones y cómo mantenerlo, probarlo y evolucionarlo.

Mensaje final

Este ejercicio tiene como finalidad brindarles una referencia práctica del nivel técnico y de las competencias que deben fortalecer antes de su prueba de desempeño.

El contexto, las entidades y las reglas de negocio son diferentes a los de la evaluación oficial. Sin embargo, los conocimientos que deberán aplicar corresponden a los temas trabajados durante el módulo.

Aprovechen este espacio para identificar dificultades, investigar, reforzar conceptos y ganar autonomía en la construcción de soluciones backend profesionales.

El verdadero indicador de preparación será que puedan construir, explicar, probar y defender técnicamente su propia solución.